

Complete.PIF.Analysis.PartOne.lasting.cluster_documenta

Fred Cudhea

08 July, 2025

Documentation for R script: Complete.PIF.Analysis.PartOne.lasting.cluster.r, which estimates the PIFs for all diet-disease pairs of interest, using probabilistic sensitivity analysis to propagate uncertainty (Which just means we use monte carlo simulations to calculate the PIFs multiple times). The general idea for this script is we traverse through each diet-disease pair and each subgroup, and calculate the PIF 1000 times. Inputs are drawn randomly from a distribution and that uncertainty propagates into the results.

Please review text on methods (this code is just implementation of methods) to better understand what is going on (e.g. appendix of this paper: <https://pmc.ncbi.nlm.nih.gov/articles/PMC5702980/> (<https://pmc.ncbi.nlm.nih.gov/articles/PMC5702980/>), but any PIF paper where I am co-author should have similar description).

First, we create various R objects where we store final output, as well as intermediate outputs needed to calculate the PIFs.

```
#Complete.PAF.analysis.partOne.r
#setwd(paste("C:\\Users\\Fred Cudhea\\Box Sync\\new Life\\USmortality\\mort", year.vec.string, "us" , sep="")) ####gms
#setwd(paste("/cluster/home/f/c/fcudhe01/CRA/mort", year.vec.string, "us/", subfolder , sep=""))
#setwd(paste("C:\\Users\\jliu28\\Cancer R01\\CRA modelling_Fangfang\\Independent effect\\SSB effect" , sep="")) ####gms
#setwd(paste("C:\\Users\\jliu28\\Cancer R01\\CRA_FF\\Incidence" , sep="")) ####gms
#setwd(paste("C:\\Users\\Fred Cudhea\\Box Sync\\Juju and Fred\\Incidence Code", sep=""))
#setwd(paste("C:\\Users\\jliu28\\Box Sync\\Juju and Fred\\Incidence Code", sep=""))
#setwd(paste("/Users/fcudhe01/Box Sync/Juju and Fred/Incidence Code", sep=""))
#setwd(paste("C:\\Users\\fcudhe01\\Box Sync\\new Life\\Cancer-CRA-organized\\inputs", sep=""))
#setwd("C:\\Users\\fcudhe01\\Box\\Lasting_aim_3\\data\\in\\example_cra")
#setwd(paste("C:\\users\\", Sys.info()[7], "\\Box\\Lasting_aim_3\\model development\\data_new\\in\\TEMP", sep=""))
#setwd("Inputs")
mort.list<-list() ##List of morts draws, one element per dietary factor
#observed.mort.list<-List()

# BROOKE IS AWESOME

PIF.list<-list() # a list of lists, giving the calculated PAFs by riskfactor, disease, strata, and iteration

PIFs<-list() ## above, a list of dataframes
samp_beta<-list()

data<-c()
q<-c()
qsx<-c()
qsx.alt<-c()
mtc<-c()
mtc.alt<-c()
x<-c()
x.alt<-c()
qsy<-c()
tmmtc<-c()
y<-c()
delat<-c()
delat.alt<-c()
p<-c()
p.alt<-c()
p.pp<-c()
p.pp.alt<-c()

delat<-list()
delat.alt<-list()
rr.list<-list()
rr.list.alt<-list()
p.rr<-list()
p.rr.alt<-c()
p<-list()
p.alt<-list()
pif<-replicate(num.diseases, matrix(NA, nsim2, nsim1), simplify=F)
mort<-replicate(num.diseases, matrix(NA, nsim2, nsim1), simplify=F)
observed.mort<-replicate(num.diseases, matrix(NA, nsim2, nsim1), simplify=F)
#observed.mort<-split(x=observed.mort.draws, f=observed.mort.draws$disease)

#paf<-List() ##<-matrix (NA,nsim2, nsim1) # will need to change if the exposure dataset is longer than 5000 rows
#mort<-List() ##<-matrix (NA,nsim2, nsim1) ####gms empty matrix for mortality estimations

#pif<-replicate(num.diseases, matrix(NA, nsim2, nsim1), simplify=F)

final_mat_pif<- matrix (NA, nsim2, 12)
final_mat_mort<- matrix (NA, nsim2, 12) ####gms matrix with final mortality estimates

m<-list()
s<-list()
m_mort<-list()
s_mort<-list()
```

We are going to be using nested for loops to traverse through everything we need. Don't get lost, pay attention! Here is a primer on nested for loops: <https://www.spsanderson.com/steveondata/posts/2025-03-10/#> (<https://www.spsanderson.com/steveondata/posts/2025-03-10/#>):~:text=A%20nested%20for%20loop%20is,loop%20runs%20completely%20(all%20iterations)

The highest level of the nested for loop is for looping through the risk factors of interest. We'll call it the r loop.

```
#####
# First Loop for Each Risk Factor #####
#####
# this loop runs for each risk factor in the vector rfvec

# debug
# r = 6
# rfvec[6]

for (r in 1:length(rfvec_cra)){ #####gms Start risk factor Loop
```

Within the r loop, first extract inputs specific to risk factor r, which includes RRs, risk factor to BMI and risk factor to SBP effects , and the **TMRED**. Recall that the data below are read in / extracted in LASTING_cluster_w_masterinput.r in the **## CRA-SPECIFIC INPUTS ##** section.

```
# vector of age-sex-race specific files to be read in; for dietary risks only one file is in the vector

rr = subset(rrtotal, RF == rfvec_cra[r])      # subset the relative risks dataset to the risk factor of interest
#impact_factor<-subset(impact_factors_total, RF == rfvec[r]) ##Likewise for impact factor
#print(head(rrtotal))
#print(rr)
#print(r)
print(rfvec_cra[r])
#rr$gender[rr$sex == 2] = "F"
#rr$gender[rr$sex == 1] = "M"
#rr$female[rr$sex == 2] = 1
#rr$female[rr$sex == 1] = 0

food.to.BMI<-subset(Food.to.BMI.effects, food_group==rfvec_cra[r])
food.to.SBP<-subset(Food.to.SBP.effects, food_group==rfvec_cra[r])

# theoretical minimums: *** Specific for each rf *****
## iterate here to go through all the risk factors
theominsub = subset(theomin, Risk_factor==rfvec_cra[r])
mu_tmrd = theominsub$TMRED
sd_tmrd = theominsub$SD
```

Within the r loop, we are going to start another for loop, which we'll call the l loop. This is a bit of obsolete legacy code, because it's for looping through multiple data sets, which we never do (we always have one exposure dataset, and opt to rerun the script if we have multiple datasets). That's why code to read in the exposure data is within this loop.

```
#data0 = read.csv(file=file_m[l], header=T)
data0<-expos
# data0 will be the exposure file

data01 = subset(data0, diet==rfvec_cra[r])
# data01 will be the subset of a exposure file for each risk factor
## iterate here to go through all the risk factors.

#data1 = data01[order(data01$year, data01$iso3, data01$sex, data01$age),]

#if(identical(covar.vec, c("Age", "Sex", "Race"))){
#{
# data1 <- data01[order(data01$female, data01$age, data01$race),]
#}

#if(identical(covar.vec, c("Age", "Sex", "Race"))){
#{
# data1 <- data01[order(data01$subgroup),]
#} ##The purpose of the ordering is to make the risk factor / death data input file consistent with the rr file. Turns out
##In any case, since rr file is ordered by subgroup, I am just going to change the code to order the rf data to be order
#stange that we didn't see more stark difference sin other foods besides ssb.

# data1 will be the ordered data01
# make sure that the rf data is in the same order as the rr data
```

It's possible for the intake means used to calculate PIFs to be negative (If we are drawing from a normal distribution and the estimate for mean intake is very low). Should be rare but given how many times we draw sims (1000 times for each risk factor-subgroup combo) for mean intake, it's very possible that it happens once in a while. The following is creating a place to store how often this happens (this is internal info for debugging purposes, not output that you'd want to report).

```
count.mm.tozero<-0
count.mm.alt.tozero<-0
total.count<-0
```

Within the r-l loop, we are starting a new loop, which we will call the i loop. Here, we are traversing through each subgroup (for each dietary factor, since we are nested within the r-l loop).

```
#for (i in 1:length(data1$exposmean)) {   #####gms Start of loop that goes through each age/sex/country row for a given ri
# i=1

for (i in 1:nsim2) {
  # This loop will be repeated for the length of age/sex subgroups for each risk factor
```

We can't compute PIFs unless we have a SD value for the risk factor. Therefore we skip a subgroup if SD value is NA by having all the PIF calculations done with this if statement

```
if(!is.na(data1$sd[i]))
{
```

Still within the i-loop: Extract RR for subgroup i. (Within an if statement to ensure subgroup is defined in rr file)

```
if(identical(covar.vec, c("Age", "Sex", "Race")))
if(!is.null(rr$subgroup))
{
  #subset.rr = subset(rr, rr$age == data1$age[i] & rr$female==data1$female[i] & rr$race == data1$race[i])#####gms04241
  subset.rr = subset(rr, rr$subgroup == data1$subgroup[i])#####gms04241--add race strata here

  #subset.impact <- subset(impact_factor, impact_factor$age == data1$age[i] & impact_factor$female==data1$female[i] &
  }
  #<-should add [i]
  #<-should add [i] here?
```

Still within the i-loop: create some lists to store some intermediate inputs and outputs.

```
#      print(r)
#      print(i)
#      print(subset.rr)
#      print(rr)
#      print("")
# subset.rr will be the rr specific to the age/sex group
beta<-list()
samp_beta<-list()
mort.samp<-list()
```

Still within the i-loop: Create draws (from normal distribution) for current and counterfactual mean intake.

```
mm1 = rnorm(n=nsim1, mean = data1$mean[i], sd= data1$se[i])
mm2 = rnorm(n=nsim1, mean=data1$CF_mean_intake[i], sd=data1$CF_se_intake[i])
```

Still within the i-loop: we recode some variables (age and sex) and simulate draws for certain inputs relevant for mediated effects.

```
#get prevalence of BMI > 25
overweight.prev<-rnorm(n=nsim1, mean=data01$overweight_rate[i], sd=data01$overweight_rate_se[i])/100
overweight.prev[overweight.prev<0]<-0
overweight.prev[overweight.prev>1]<-1

## define the variables necessary for modifying the linear effect of Na on BP (age, race, htn..)
#region = SSB.BMI.mort$gbd_region[i] not used, as far as I can tell.
#agemid = merged.data$age[i]+5
#agemid = data01$age[i]+5##50 ##for testing purposes only. revert back to agemid = data01$age[i]+5 for age/sex version
#if(agemid==80) agemid<-87.5
#^instead of defining agemid in the cose, best to define it in the inputs
gender = data01$female[i]+1 ##gender = female + 1 in this hypertension model
#income = SSB.BMI.mort$crudincome[i] #assuming high income is coded as one here. (This is a country level covariate so
#SBP = data01$sbp_mean[i]
agemid<-data01$age.mid[i] ##agemid predefined in input data

#logitprev = 0.107*SBP + -0.45*(1) + 0.004*agemid + 1.24*gender - 22.7
#prev = exp(Logitprev)/(1+exp(Logitprev))
#We now use hypertension prevalence obtained from nhanes
htn.prev <- rnorm(n=nsim1, mean=data01$hbp[i], sd=data01$hbp_se[i])
htn.prev[htn.prev<0]<-0
htn.prev[htn.prev>1]<-1

#similarly, we use black prevalence obtained from nhanes
black <- rnorm(n=nsim1, mean=data01$nhb[i], sd=data01$nhb_se[i])
black[black<0]<-0
black[black>1]<-1

## use proportion of high sbp (over 115) to get overall sodium to sbp effect for each subpopulation
high.SBP.prev<-rnorm(n=nsim1, mean=data01$highSBP_rate[i], sd=data01$highSBP_rate_se[i])/100
high.SBP.prev[overweight.prev<0]<-0
high.SBP.prev[overweight.prev>1]<-1

## simulating and sample LogRRs
```

Nested within the r,i loop, we are now starting the j-loop, looping through the diseases/pathways (for subgroup i and exposure r).

```
#      num.diseases
#      diseases.vec
#      j<-7

for(j in 1:num.diseases)
{
```

Within the j-loop: Simulate draws for the RRs (per unit increase in log(RR) to be precise).

```
#paf[[j]]<-matrix(NA,nsim2, nsim1)

#mort[[j]]<-matrix(NA,nsim2, nsim1)

#      print(j)
#      print(nsim1)
#      print(subset.rr[[diseases.vec[j]])
#      print(subset.rr[[diseases.vec.se[j]])
beta[[j]]<-rnorm(n=nsim1, mean=subset.rr[[diseases.vec[j]]], sd=subset.rr[[diseases.vec.se[j]])
samp_beta[[j]]<-sample(beta[[j]], nsim1)
#      print(beta[[j]])
#      print(r)
#      print(i)
#      print(j)
#      if(is.na(beta[[j]][1]))
#      stop()

#Legacy code that moves change in log-rr to 0 if on the opposite sign of what it's "supposed to be"
#if(subset.rr$protective==1) {samp_beta[[j]][samp_beta[[j]] > 0] = 0} else {samp_beta[[j]][samp_beta[[j]] < 0] = 0}

#cat("diseases.vec[j]: ", diseases.vec[j], "\n")
#cat("subset.rr[[diseases.vec[j]]]: ", subset.rr[[diseases.vec[j]]], "\n")
#cat("samp_beta: ", samp_beta[[j]], "\n\n")
```

Within the j-loop: Simulate draws for effect of risk factor on BMI for normal weight and overweight. Use these, plus draws for overweight prevalence to get draws for overall effect of riskfactor on BMI for the subgroup.

```
##From Gita's SSB code
#####
### STEP 2a: DRAW AN ESTIMATE LINEAR EFFECT OF SSB ON BMI ###
#####

# We will determine the increase in BMI per serving/day increase in SSB
# As of 04.04.12 we will use the linear effects from 3 cohorts:
# for BMI <= 25 kg/m2 use 0.10
# for BMI > 25 kg/m2 use 0.23

beta.lin.low = rnorm (n=nsim1, mean=food.to.BMI$effect_normal_mean, sd=food.to.BMI$effect_normal_mean_se)
# print(beta.lin.low)
# if(is.na(beta.lin.low[1])) stop()
samp.beta.lin.low = sample (beta.lin.low, nsim1)
#samp.beta.lin.low[samp.beta.lin.low < 0] = 0

beta.lin.high = rnorm (n=nsim1, mean=food.to.BMI$effect_overweight_mean, sd=food.to.BMI$effect_overweight_mean_se)
samp.beta.lin.high = sample (beta.lin.high, nsim1)
#samp.beta.lin.high[samp.beta.lin.high < 0] = 0

food.to.bmi.effect<-beta.lin.high*overweight.prev+beta.lin.low*(1-overweight.prev)
#####
#####
```

Within the j-loop: Simulate draws for main and interaction effects of dietary factor on SBP. Use these, and the draws for the proportion of Black people and prevalence of hypertension simulated earlier, to create draws for the overall effect of dietary factor r on SBP for subgroup i.

```
#####
### STEP 2a: DRAW AN ESTIMATE LINEAR EFFECT OF SODIUM ON SBP ###
#####

## Sodium effects on blood pressure

#####
#####
## Main effects##
#####

#maineffect = rnorm(n=nsim1, mean=1.62, sd= ((2.25-0.994)/3.92) )
maineffect = rnorm (n=nsim1, mean=food.to.SBP$main.effect.mean, sd=food.to.SBP$main.effect.se)

#####
## Interaction by age ##
#####

#ageeffect = rnorm(n=nsim1, mean = 0.0457, sd = ((0.071-0.0204)/3.92))
age_effect = rnorm(n=nsim1, mean=food.to.SBP$age.effect.mean, sd=food.to.SBP$age.effect.se)

#####
## Interaction by race ##
#####

#raceeffect = rnorm(n=nsim1, mean = 1.08, sd = ((2.11-0.304)/3.92))
race_effect = rnorm (n=nsim1, mean=food.to.SBP$race.effect.mean, sd=food.to.SBP$race.effect.se)

#####
## Interaction by hypertensive status ##
#####

#htneffect = rnorm(n=nsim1, mean = 0.815, sd = ((1.56-0.052)/3.92))
htneffect = rnorm (n=nsim1, mean=food.to.SBP$htn.effect.mean, sd=food.to.SBP$htn.effect.se)

#####

# here assign a value to the linear effect of Na on BP, depending on age, region and htn status

# age effect
#adjeffect.a <- maineffect + (agemid-50)*age_effect
#print(agemid)
#if (agemid > 50 & agemid < 70 ) {
# adjeffect.a = maineffect + (agemid-50)*ageeffect
#} else if (agemid <50) {
# adjeffect.a = maineffect - (50-agemid)*ageeffect
#} else
# adjeffect.a = maineffect
if(agemid>70)
{
  adjeffect.a[agemid>70] <- (70-50)*age_effect ##for people over age 70, we use the same effect as people at age 70.
  ##In other words, the relationship between age and linear effect of Na on BP is assumed to be flat after age 70.
}else{
  adjeffect.a <- maineffect + (agemid-50)*age_effect
}

# race effect
adjeffect.a.r = adjeffect.a + race_effect*black

# effect of hypertension (weight by proportion hypertensive)
adjeffect.a.r.h = htn.prev*(adjeffect.a.r + htneffect) + (1-htn.prev)*(adjeffect.a.r)

food.to.sbp.effect<-adjeffect.a.r*high.SBP.prev #+0*(1-high.sbp.prev)
##This seems weird but in fact, similar to the original version.
#in original, we calculate adjusted effect based on race/hypertension prevalence, etc...
#But then only use this effect if mean_sbp > 115. If mean_sbp < 115, we have no effect at all.
#instead of doing this, here, we're going to be a little less crude and multiply effect by proportion above 115.
#So borderline group will only get half the effect instead of all or none, etc...
```

Still within the j-loop: extract mortality/incidence draws for disease j (and subgroup i) and store them into the observed.mort list created earlier.

```

###Get and save, observed mort draws
#subset.observed.mort.draws<-subset(observed.mort.draws, observed.mort.draws$disease==diseases.vec[j] &
#                                     observed.mort.draws$agecat == data1$agecat[i] & observed.mort.draws$female==da

subset.observed.mort.draws<-subset(observed.mort.draws, observed.mort.draws$disease==diseases.vec[j] &
                                   observed.mort.draws$subgroup == data1$subgroup[i])

observed.mort.draws.start.point<-which(names(subset.observed.mort.draws)=="1")
observed.mort.draws.end.point<-which(names(subset.observed.mort.draws)==paste(nsim1))
observed.mort[[j]][i,<-t(subset.observed.mort.draws[,observed.mort.draws.start.point:observed.mort.draws.end.point]

```

Now, still within the j-loop, we start the k-loop, which loops through all simulated draws (for a given disease/pathway and risk factor) to calculate the PIF nsim1 times (Recall nsim1 is set to 1000 in LASTING_cluster_w_masterinput.r, but you can set it whatever you want. For testing purposes, you'll probably want to set it to something much smaller initially).

```

#           k<-1

for (k in 1:nsim1){      #####gms1 Start Simulation Loop

```

Within the k-loop, extract k-th mean and standard deviation for both current and counterfactual intake (making sure to set them to slightly above zero for the rare instances that they end up being at or below zero). Derive gamma parameters based on extracted means and standard deviation. Recall that the model assumes the usual intake distribution for all food groups follow a gamma distribution, defined by two parameters: shape and rate.

```

mm<-(mm1[k])          ## the mean for each age/sex group
mm.alt<-(mm2[k])

sdd = data1$sdd[i]      #####gms          ## *** ** Specific for each RF**** data from mean to sd regression based on
sdd.alt = data1$CF_sd_intake[i]

#count<-0
#if(mm<=0)
#{
# mm<-0.00001  ##can't have negative values or 0 values for shape and rate so in this situatin we set mean to ne
# count<-count+1
#}
## in practice, this will mean that the likelihood is concentrated with incredibly low numbers.
##In these situations, we may overestimate the paf since current intake is basically 0 for everyone in these cases
##which is an obvious underestimation of true intake
##You won't get absolute 0 when using normal distribution with truncated zeros, but you're still most likely
## considerably underestimating true intake (since over half is assumed to 0 intake)
## In any case, this scenario should be incredibly rare, so in practice, not a big issue (hopefully)
## And if it is a big issue, we essentially have an issue with sample size (way too small)
#if(sdd<=0)
#{
# sdd<-0.00001  ##can't have negative values or 0 values for shape and rate so in this situatin we set standard
# count<-count+1
#}

total.count<-total.count+1
if(mm<=0)
{
mm<-0.001  ##can't have negative values or 0 values for shape and rate so in this situatin we set mean to near
count.mm.tozero<-count.mm.tozero+1
}
#count.mm.alt.tozero<-0
if(mm.alt<=0)
{
mm.alt<-0.001 ##intervention.effect[k]  ##can't have negative values or 0 values for shape and rate so in this
sdd.alt = sdd*mm.alt/mm
count.mm.alt.tozero<-count.mm.alt.tozero+1
}

shape<-mm^2/sdd^2
rate<-mm/sdd^2
shape.alt<-mm.alt^2/sdd.alt^2
rate.alt<-mm.alt/sdd.alt^2

```

Still within the k-loop: we are going to calculate the PIFs using Reimann sums, meaning we are going to slice up the gamma distribution into small slices, multiply each slice by RR(x), and then sum them all up. First step then, is to generate quantile values to mark up where to slice up the exposure distributions. Note that that these values will be used for RR(x) as well. Essentially, we are determining the values for "x" and "y" in our PIF formula.

First, we generate "q" which is just slicing up a standard normal 121 times in .1 size increments. The smaller the width of your slice, the more precise Reimman integration is going to be. We are basically saying .1 is thin enough for us, and -6 to 6 is a wide enough range to cover the standard normal (which it most certainly is, covering more than 99.999999% of the probability mass). From there, we derive "y", the quantiles for the TMRED. "x" the quantiles for the current exposure distribution following a gamma, and "x.alt" the quantiles for the counterfactual exposure distribution.

```
q<-seq(-6,6,by=.1)          ## generate the "slices" of standard normal distribution between -6 and 6 by 0.1
mtc<-rep(mm, times=121)      ## vector with 121 repetitions of mean for the age/sex group of interest (121 =
tmmtc<-rep(mu_tmrd, times=121) ## vector with 121 repetitions of the theoretical minimum for mean
#qsx<-sdd*q                  ## vector multiplying sd for the age/sex group of interest by each "slice"
#x=(mtc+(qsx))                ## vector summing 121 repetitions of mean with the "sliced" sd
qsy<-sd_tmrd*q                ## vector multiplying theoretical minimum sd by slices
y<-(tmmtc+(qsy))              ## vector summing 121 repetitions of theoretical minimum mean with the "sliced"

#x<-qgamma(pnorm(y, mean=mu_tmrd, sd=sd_tmrd),
#          shape=shape, rate=rate)

#x<-qgamma(pnorm(mtc+(sdd*q), mean=mm, sd=sdd),
#          shape=shape, rate=rate)

x<-qgamma(pnorm(q, mean=0, sd=1),
          shape=shape, rate=rate)
x.alt<-qgamma(pnorm(q, mean=0, sd=1),
              shape=shape.alt, rate=rate.alt)

if(is.na(sum(x.alt)))
{
  print(mm)
  print(sdd)
  print(mm.alt)
  print(sdd.alt)
}
#print(head(x))
#print(shape)
#print(rate)
# cat("\n")
#This will get you the same x values as line commented out above using normal tmred distribution or the
##incorrect "current" distribution assuming normality.
## Unless a proper tmred distribution exists (which doesn't for pmeat and others where
##0 sd is assumed), I can't use "y" to define x so I use q instead (it's all the same anyway)

#                                delat<-(y-x)/subset.rr$RRunit
#                                print(i)
#                                print(k)
#                                print(sdd)
#                                if(is.na(mm))
#                                stop()
#p.pp<-dnorm(x, mean=mm, sd =sdd, log = FALSE)*(1*sdd)
#p.pp<-pgamma(x, shape=shape, rate=rate)
```

Still within the k-loop: Get the area under the curve for each slice. We can calculate this by taking the cumulative distribution function (that's what the pgamma function does) at q-th quantile and the (q-1)th quantile and taking the difference. Note that x.shifted vector is just x but shifted to the right one space, and having the first element be zero. Likewise for x.shifted.alt. So the q-th element in x.shifted is the (q-1)-th element in x. Using x.shifted lets us easily calculate the area under the curve for each slice in one line.

```
x.shifted<-c(0, x[-length(x)]) ##same as x but lowest value is 0 and highest value of x is omitted
x.shifted.alt<-c(0, x.alt[-length(x.alt)]) ##same as x but lowest value is 0 and highest value of x is omitted

##this is used to get probability between points in x-values
p.pp<-pgamma(x, shape=shape, rate=rate)-pgamma(x.shifted, shape=shape, rate=rate)
## determine the prevalence for each slice of each age-sex group: multiply the density of a normal distribution for
p.pp.alt<-pgamma(x.alt, shape=shape.alt, rate=rate.alt)-pgamma(x.shifted.alt, shape=shape.alt, rate=rate.alt)

#x[x<0]<-0
```

Still within the k-loop: Next we compute RR(x) for each slice. What the RR function is depends on which pathway we are on (direct, BMI-mediated, SBP-mediated), hence the ifelse statements. Recall that we essentially treat each foodgroup-pathway combination as its own exposure, since they have different relative risks. Note that the food to bmi/sbp effect units are baked into the code (That's where the 5 and 10 come in). If RR(x) ever ends up being less than 1 (meaning the shift becomes harmful rather than helpful for health. Possible in tail ends of skewed distribution where P(x) might exceed tail end of normally distributed TMRED), we set it to 1. TMRED means minimum risk, so there should be no increase in moving to TMRED.

```

if(j %in% grep(diseases.vec, pattern="medBMI"))
{
  ##defining delat and defining rr.List[[j]] are the two things that are different for mediated effects and direct

  ##For delat, the key difference is that RRunit is not the same, since this is the effect on risk per unit of BMI
  ##Actually, the way we do it now, the food.to.bmi.effect does not have a fixed unit, so
  ##we divide by RRunit, as well as 5kg/m^2. e.g: if food.to.BMI effect is 100(g/d)(kg/m^2), and delta is in g,
  ##then you want have (g/d)/(5kg/m^2)*(kg/m^2 / 100 g/d)
  delat[[j]]<-(x-y)/5/subset.rr$RRunit
  delat.alt[[j]]<-(x.alt-y)/5/subset.rr$RRunit

  ##for rr.List[[j]], the key difference is that we must multiply by beta.lin.low or beta.lin.high, depending on w
  ## is for that group. beta.lin is how much BMI increases per one(?) unit of increase in SSB.

  ##if(data1$bmi_mean[i]<=25)
  #{
  # rr.List[[j]]<-exp(delat[[j]]*samp_beta[[j]][k]*samp.beta.lin.Low[k])
  #}
  #else
  #{
  # rr.List[[j]]<-exp(delat[[j]]*samp_beta[[j]][k]*samp.beta.lin.high[k])
  #}

  rr.list[[j]]<-exp(delat[[j]]*samp_beta[[j]][k]*food.to.bmi.effect[k])
  rr.list.alt[[j]]<-exp(delat.alt[[j]]*samp_beta[[j]][k]*food.to.bmi.effect[k])

}else if(j %in% grep(diseases.vec, pattern="medSBP")){

  delat[[j]]<-(x-y)/10/subset.rr$RRunit
  delat.alt[[j]]<-(x.alt-y)/10/subset.rr$RRunit

  rr.list[[j]]<-exp(delat[[j]]*samp_beta[[j]][k]*food.to.sbp.effect[k])
  rr.list.alt[[j]]<-exp(delat.alt[[j]]*samp_beta[[j]][k]*food.to.sbp.effect[k])

}else
{

  delat[[j]]<-(x-y)/subset.rr$RRunit      #####gms      ## delta is the difference between the actual and theor
  delat.alt[[j]]<-(x.alt-y)/subset.rr$RRunit
  #cat("delat when we define it: ", delat, "\n\n")
  rr.list[[j]]<-exp(delat[[j]]*samp_beta[[j]][k])
  rr.list.alt[[j]]<-exp(delat.alt[[j]]*samp_beta[[j]][k])
  #cat("delat: ", delat, "\n")
  # rr.ihd [delat>0]<-1      ## set attributable risk to 1 if actual exposu
}

#rr.List[[j]][delat[[j]]<0]<-1
rr.list[[j]][rr.list[[j]]<1]<-1
rr.list.alt[[j]][rr.list.alt[[j]]<1]<-1
##when using rr.list instead of delat to define what to force to 0. The ifelse statement is not necessary,
if(diseases.vec[j]=="SC" & rfvec[r]=="sodium")
{
  cat("delat[[j]]: ", delat[[j]], "\n")
  cat("rr.list[[j]]: ", rr.list[[j]], "\n")
}

```

Still within the k-loop: Multiply RR by the area of the slices and sum up. Calculate the PIF! Store the value in list of data frames created earlier.

```

#p.rr[[j]]<-p.pp*(rr.list[[j]]-1)          ## numerator of the paf calc for each "slice"
#p[[j]]<-sum(p.rr[[j]], na.rm=TRUE)        ## sum the numerator vector across all "slices"
#paf[[j]][i,k]<- p[[j]]/(p[[j]]+1)         ## actual paf calculation, generates a matrix of i rows and k columns
#                                         ihd.mort = rnorm(n=nsim1, mean = data1$ihdmort[i], sd = ((data1$uciihdmort[i]-data1$lci
#                                         ihd.mort.samp = sample(ihd.mort, 1)          #####gms sample from mortality distributi
#mort.samp[[j]]<-rnorm(n=1, mean=data1[,colnames(data1)==diseases.vec.mn[j]][i], sd=data1[,colnames(data1)==diseas
#observed.mort[[j]][i,k]<-rnorm(n=1, mean=data1[,colnames(data1)==diseases.vec.mn[j]][i], sd=data1[,colnames(data1
## Instead we just use mortality count for this particular strata
#cat("p[[j]][i,k]: ", p.rr[[j]], "\n")
#cat("paf[[j]][i,k]: ", paf[[j]][i,k], "\n")
#cat("mort.samp[[j]]: ", mort.samp[[j]], "\n")

p.rr[[j]]<-p.pp*rr.list[[j]]
p.rr.alt[[j]]<-p.pp.alt*rr.list.alt[[j]]
p[[j]]<-sum(p.rr[[j]], na.rm=TRUE)
p.alt[[j]]<-sum(p.rr.alt[[j]], na.rm=TRUE)
pif[[j]][i,k]<-(p[[j]]-p.alt[[j]])/p[[j]]

```

Still within the k-loop: There are various scenarios where the PIF must be zero theoretically, but because of computational issues, you may not get exactly zero when computing it. In these cases, we set the PIF to zero manually. Meaning, if conditions below are met (relative risk for this simulation is 0, or risk profile for counterfactual distribution somehow being greater than risk profile for current distribution, for example), we replace whatever was calculated above with zero.

```

if(p[[j]]==0)
  pif[[j]][i,k]<-0

### samp_beta is the effect of food on risk per dose for this particular iteration of the simulation
### if it's zero, that means there is no effect in changing food distribution, so the PIF is 0
### By just just setting it to 0, we can avoid complications
if(samp_beta[[j]][k]==0)
  pif[[j]][i,k]<-0

##If counterfactual is more harmful than current exposure distribution, then we can get negative numberers for PIF
##Just set it to zero in that case

# Brooke note: getting error here for r=5...

if(pif[[j]][i,k]<0)
  pif[[j]][i,k]<-0

```

Still within the k-loop: Print various inputs and outputs for debugging purposes if PIF is NA

```

if(is.na(pif[[j]][i,k]))
{
  print(r)
  print(i)
  print(j)
  print(k)
  cat("pif[[j]][i,k]", pif[[j]][i,k], "\n")
  cat("p[j]", "\n")
  print(p[j])
  cat("p.alt[j]", "\n")
  print(p.alt[j])
  cat("mm", mm, "\n")
  cat("sdd", sdd, "\n")
  cat("mm.alt", mm.alt, "\n")
  cat("sdd.alt", sdd.alt, "\n")
  cat("rr.list[[j]]", rr.list[[j]], "\n")
  cat("rr.list.alt[[j]]", rr.list.alt[[j]], "\n")
  cat("delat[[j]]", delat[[j]], "\n")
  cat("delat.alt[[j]]", delat.alt[[j]], "\n")
  cat("subset.rr$RRunit", subset.rr$RRunit, "\n")
  cat("x", x, "\n")
  cat("x.alt", x.alt, "\n")
  cat("y", y, "\n")
}

```

Still within the k-loop: Final step: Multiply the PIF to your observed mortality/incidence to attributable mortality/incidence.

```

#mort[[j]][i,k] = paf[[j]][i,k]*mort.samp[[j]]      #####gms multiply by the corresponding ith sampled mortality here
mort[[j]][i,k] = pif[[j]][i,k]*observed.mort[[j]][i,k]      #####gms multiply by the corresponding ith sampled mortality here

#cat("j: ", j, "\n")
#cat("i: ", i, "\n")
#cat("k: ", k, "\n\n")
#cat("mort[[j]][i,k]: ", mort[[j]][i,k], "\n")
#cat("delat a the end: ", delat, "\n\n")

```

Close the k-loop.

```

}
#print(count)

```

Close the j-loop.

```

}

#####gms1 End Simulation Loop (nsim1)

```

Close the if statement for making sure SD estimate exists.

```

}      #####      End if statement on whether sample size is greater than one for this group.

```

Close the i-loop.

```

}      #####gms      End Age/sex/country Loop (nsim2)

```

We haven't closed all the loops yet! We are still in the i-loop. Recall that pif is a list of data frames we created within the r-loop. Each element in the list is a dataframe (one data frame for each disease/pathway, each dataframe has simulated PIFs for each subgroup (subgroups are rows, number of simulations is columns)). Here, we are simply storing this list as the r-th element of PIF.list. So PIF.list is going to store the entirety of all the PIF simulations generated.

And then, we close the i-loop.

```

PIF.list[[r]]<-pif
}      #####gms End File Loop (nsim3)

```

We are now still in the r-loop. The rest of the code within the r-loop is about creating two sets of dataframes for each riskfactor with all simulations (for all disease/pathways and all subgroups) and making sure all rows are properly labeled. First set of data frames is for attributable mortality/incidence, second set of data frames is for PIFs (both calculated within the k-loop).

First extract subgroup info (subgroup + current and counterfactual exposure). Then create lists to store intermediate output.

```

### Here we want to put the paf and mort matrices for each outcome into one giant matrix for each risk factor that can be us
### make each outcome's matrix of mortalities or pafs into a data frame and label the columns with draw_"num"
### then merge this with a segment of the exposure/mortality dataset for identification and further calculations
### merge also with population data for further calculations
### add a column with the outcome name
### then rbind all the individual outcome dataframes together

#ccodes = read.csv(paste(CCDir, "countryregioncodesincome.csv", sep=""))
#ccodes = ccodes[,c("iso3", "wbincome")]
#print("error here?")
a = paste("mortdraw.", seq(1, nsim1, by=1), sep='')

if(identical(covar.vec, c("Age", "Sex", "Race")))
  id = data1[,c("age", "female", "race", "mean", "se", "sd", "CF_mean_intake", "CF_se_intake", "CF_sd_intake")]#,
#       "ihdmort", "lciihdmort" "uciihdmort",
#       "istkmort", "lciistkmort", "uciistkmort",
#       "hstkmort", "lcihstkmort", "ucihstkmort",
#       "diabmort", "lciidiabmort", "ucidiabmort")]
#id = merge(id, ccodes, by = "iso3")

mort.df<-list()
mort.df2<-list()

PIF.df<-list()
PIF.df2<-list()

# observed.mort.df<-list()
# observed.mort.df2<-list()
#print("error here?2")

```

Next, we will loop through mort.df and pif.df (both lists where each element is a matrix that contains the attributable mortality/incidence (the mort/incidence attributed to the population being at the current exposure distribution rather than the counterfactual exposure distribution) or PIF (potential impact fraction, aka, the proportion of mort/incidence attributable to being at the current exposure distribution rather than the counterfactual exposure distribution) sims for a disease/pathway), turning each matrix into a data frame and outcome of interest, pathway, and disease type. We then bind the subgroup level info we extracted earlier and call the new data frame mort2 (or pif2). So mort2 and pif2 are lists where each element is a data frame containing all the simulations for risk factor r and disease/pathway ii with proper labeling of rows and info specific to the disease/pathway and risk factor.

```

for(ii in 1:num.diseases)
{
  mort.df[[ii]]<-as.data.frame(mort[[ii]])
  #mort.df[[ii]]$outcome<-diseases.vec[[ii]]
  #mort.df[[ii]]$outcome_all<-disease.total.vec[[ii]]
  mort.df[[ii]]$outcome<-disease.total.vec[[ii]]
  mort.df[[ii]]$pathway<-pathway[[ii]]
  mort.df[[ii]]$disease_type<-disease.type.total.vec[[ii]]
  mort.df[[ii]]$disease_type1<-disease.type1.total.vec[[ii]]
  mort.df[[ii]]$disease_type2<-disease.type2.total.vec[[ii]]
  mort.df[[ii]]$disease_type3<-disease.type3.total.vec[[ii]]

  # check
  # mort.df[[ii]] %>% View()

  mort.df2[[ii]]<-cbind(id, mort.df[[ii]])

  PIF.df[[ii]]<-as.data.frame(PIF.list[[r]][[ii]])
  #PIF.df[[ii]]$outcome<-diseases.vec[[ii]]
  #PIF.df[[ii]]$outcome_all<-disease.total.vec[[ii]]
  PIF.df[[ii]]$outcome<-disease.total.vec[[ii]]
  PIF.df[[ii]]$pathway<-pathway[[ii]]

  # brooke edited
  PIF.df[[ii]]$disease_type<-disease.type.total.vec[[ii]]
  PIF.df[[ii]]$disease_type1<-disease.type1.total.vec[[ii]]
  PIF.df[[ii]]$disease_type2<-disease.type2.total.vec[[ii]]
  PIF.df[[ii]]$disease_type3<-disease.type3.total.vec[[ii]]

  # check
  # PIF.df[[ii]] %>% View()

  PIF.df2[[ii]]<-cbind(id, PIF.df[[ii]])

  # observed.mort.df[[ii]]<-as.data.frame(observed.mort[[ii]])
  # observed.mort.df[[ii]]$outcome<-diseases.vec[[ii]]
  # observed.mort.df2[[ii]]<-cbind(id, observed.mort.df[[ii]])
}

# } # stop here for testing purposes #####

# this is the dataframe containing all the draws for each outcome, as well as the identifying info such as iso3, age, sex,
#print("error here?3")

```

We now bind all the elements for each list together so we have one giant file with all disease-pathways in one file, by looping through each element of mort.df2 / PIF.df2 and concatenating to allmort / allPIF.

```

allmort<-mort.df2[[1]]
allPIF<-PIF.df2[[1]]
# allObservedmort<-observed.mort.df2[[1]]
for(jj in 2:num.diseases)
{
  allmort<-rbind(allmort, mort.df2[[jj]])
  allPIF<-rbind(allPIF, PIF.df2[[jj]])
  # allObservedmort<-rbind(allObservedmort, observed.mort.df2[[jj]])
}
allmort$riskfactor = rfvec_cra[r]
allPIF$riskfactor = rfvec_cra[r]
# allObservedmort$riskfactor = rfvec[r]

allmort<-as.data.frame(allmort)

```

"n" is the number disease/pathways. Used later when summarizing results but defined here (for convenience? I'm not sure)

```

#mediated<-grep("_med", allmort$outcome)
n<-dim(allmort)[1]/(n.mediates.effects+1)

```

One last thing before we close the r-loop. Store the data frame allmort (attributable mortality/incidence sims for all subgroups and disease/pathways for riskfactor r) and allPIF (PIF sims for all subgroups and disease/pathways for riskfactor r) as element r in lists "mort.list" and "PIFs" respectively. So we'll have all relevant output results housed in these two objects (as opposed to scattered r objects, or r*n objects, etc...). Useful because it will make code for later tasks easier to read and write.

```
#allmort<-rbind(allmort, Allcancer_direct, Allcancer_medBMI, Allcancer_sumcancer_direct, sumcancer_medBMI, sumcancer, ECPLu
#
#      MPLCPlusLarynx_direct, MPLCPlusLarynx_medBMI, MPLCPlusLarynx)
mort.list[[r]]<-allmort
PIFs[[r]]<-allPIF
# observed.mort.List[[r]]<-allobservedmort
#write.csv(allmort, paste(ResultsDir,file=paste(rfvec[r], "_mortdraws_AgeSex_2010.csv",sep='')), row.names=F)
```

End the r-loop. We are almost done!

```
} ###gms End risk factor loop

# Brooke: STOP
```

Bind all the data frames in "mort.list" and "PIFs" into one data frame (in a cringy old-fashioned way, but it works). Also, update some column names for clarity.

```
all.mortdraws<-mort.list[[1]]
all.PIFs<-PIFs[[1]]
#all.observed.mort<-observed.mort.list[[1]]
for(r in 2:length(rfvec_cra))
{
  all.mortdraws<-rbind(all.mortdraws, mort.list[[r]])
  all.PIFs<-rbind(all.PIFs, PIFs[[r]])
  # all.observed.mort<-rbind(all.observed.mort, observed.mort.list[[r]])
}
names(all.mortdraws)[(1+length(covar.vec)):(6+length(covar.vec))]<-c("mean (food)", "se (food)", "sd (food)", "mean (food, cou
names(all.PIFs)[(1+length(covar.vec)):(6+length(covar.vec))]<-c("mean (food)", "se (food)", "sd (food)", "mean (food, counterf
#names(all.observed.mort)[(1+length(covar.vec)):(3+length(covar.vec))]<-c("mean (food)", "se (food)", "sd (food)")
```

Save the files into CSVs, and then delete them from R workspace (since they are large).

```
#setwd("..")
#setwd("Outputs")
#setwd("C:\\Users\\fcudhe01\\Box\\Lasting_aim_3\\data\\out\\example_cra")
#setwd(paste("C:\\Users\\", Sys.info()[7], "\\Box\\Lasting_aim_3\\model development\\data_new\\out", sep=""))
#setwd(paste("C:\\Users\\fcudhe01\\Box Sync\\new Life\\Cancer-CRA-organized\\outputs", sep=""))
write.csv(x=all.mortdraws, file=paste(output_location, diet_pattern, "/all.incidence.draws_", year.vec.string, "_", covar.vec.
write.csv(x=all.PIFs, file=paste(output_location, diet_pattern, "/all.PIFs_", year.vec.string, "_", covar.vec.string, "_", die
#write.csv(x=all.observed.mort, file=paste("all.observed.incidence.draws_", year.vec.string, "_", covar.vec.string, ".csv", se

rm(all.mortdraws, all.PIFs)
```